

Prof. Stefan Göller
Anna Maiworm

Einführung in die Informatik

WiSe 2025/2026

Hausaufgabenblatt 10, Abgabe am 26.01.26 um 12:00 Uhr

Allgemeine Hinweise

- Wenn die Hausaufgabe exakte Benennungen von Dateien, Funktionen, Variablen, etc. vorgibt, dann müssen Sie sich an diese Vorgabe halten. Falls nicht wird Ihre Abgabe nicht oder nur mit Punktabzügen korrigiert.
- Gleiches gilt, wenn die Aufgabe konkrete Anweisungen für `input()` oder `print()` vorgibt.
- Ihr Programm darf keine zusätzlichen `print()` Befehle enthalten, welche in der Aufgabe nicht gefordert sind. Ist eine Ausgabe mit `print()` gefordert, so geben Sie, sofern nicht anders vorgegeben, nur das geforderte Ergebnis und **keinen** zusätzlichen Text aus.
- Benutzen Sie **keine Python-Imports**, außer dies ist in der Aufgabenstellung explizit erlaubt.
- Beachten Sie die Anweisungen, die ggf. zusätzlich in den jeweiligen Aufgaben gegeben sind.
- Geben Sie für jede Aufgabe eine separate Datei ab. Benennen Sie diese Datei als `bXXaY.py`, wobei XX die Blattnummer und Y die Aufgabennummer ist.
- Schreiben Sie die Namen aller Gruppenmitglieder als Kommentar in die erste Zeile jeder Datei.
- Programmcode muss möglichst einfach und gut lesbar sein, um die Korrektur zu erleichtern. Code, der nicht oder nur schwer lesbar ist, kann zu Punktabzug führen.

Hausaufgabe 1 (5=1+2+2 Punkte):

Definieren Sie die folgenden Funktionen. Der Rumpf jeder der Funktionen darf dabei nur aus einer Zeile bestehen.

- (a) Die Funktion `todigitlist` erwartet einen String, der nur Ziffern enthält, und soll diese Ziffern als Liste von Integern zurückgeben.

Beispiele:

<code>""</code>	$\rightarrow []$	
<code>"68423"</code>	$\rightarrow [6, 8, 4, 2, 3]$	

- (b) Die Funktion `sum_unique_even_squares` erwartet eine Liste von Integern und soll die Summe der Quadratzahlen der geraden Zahlen zurückgeben. Wiederholte Zahlen sollen nur ein Mal zur Summe beitragen.

Beispiele:

<code>[]</code>	$\rightarrow 0$	
<code>[1, 2, 3, 4]</code>	$\rightarrow 20$	$(= 2^2 + 4^2)$
<code>[1, 2, 3, 4, 2]</code>	$\rightarrow 20$	$(= 2^2 + 4^2)$
<code>[1, -2, 3, 4, 2]</code>	$\rightarrow 24$	$(= (-2)^2 + 4^2 + 2^2)$

- (c) Die Funktion `alt_digit_sum` erwartet einen nicht-negativen Integer n und gibt die alternierende Quersumme von n zurück.

Beispiele:

<code>12</code>	$\rightarrow -1$	$(= 1 - 2)$
<code>52876</code>	$\rightarrow 10$	$(= 5 - 2 + 8 - 7 + 6)$

Hinweis: Die Funktion `sum` berechnet die Summe der Einträge einer übergebenen Liste. Zum Beispiel liefert `sum([1, 2, 3])` die Rückgabe 6.

Hausaufgabe 2 (2 Punkte):

Schreiben Sie eine Funktion `alph_dict`, welche einen Integer i zwischen 1 und 26 erwartet und mittels List- und Dictionary-Comprehensions ein Wörterbuch der folgenden Form erstellt und zurückgibt:

Die Schlüssel des Wörterbuches sind die ersten i Ganzzahlen, angefangen bei 1. Der zum Schlüssel s zugehörige Wert ist die Liste der ersten s Kleinbuchstaben, in umgekehrter Reihenfolge.

Der Rumpf der Funktionen darf dabei nur aus einer Zeile bestehen.

Beispiel: `alph_dict(6)` gibt ein Wörterbuch der folgenden Form zurück:

`{1: ["a"], 2: ["b", "a"], 3: ["c", "b", "a"], 4: ["d", "c", "b", "a"],`
`5: ["e", "d", "c", "b", "a"], 6: ["f", "e", "d", "c", "b", "a"]}`

Hausaufgabe 3 (5 Punkte):

Schreiben Sie eine Funktion `all_mod_sum`, die eine Ganzzahl $1 \leq k \leq 10$ und eine Ganzzahl $l \geq 0$ erwartet und ein Wörterbuch zurückgibt, welches jeder Zeichenkette der Länge l über dem Alphabet $\{ '0', '1', \dots, 'k-1' \}$ die Quersumme seiner Zeichen modulo k zuordnet.

Definieren Sie dazu eine Hilfsfunktion `words`, welche mittels Backtracking alle benötigten Wörter der Länge l über dem Alphabet $\{ '0', '1', \dots, 'k-1' \}$ berechnet und als Liste zurückgibt.

Beispiel: Für $k = 2$ lautet das Alphabet $\{ '0', '1' \}$. Der Aufruf `all_mod_sum(2,5)` soll das Wörterbuch mit folgenden Schlüssel-Wert Paaren zurückgeben:

Key	Value	Key	Value	Key	Value	Key	Value
'00000'	0	'01000'	1	'10000'	1	'11000'	0
'00001'	1	'01001'	0	'10001'	0	'11001'	1
'00010'	1	'01010'	0	'10010'	0	'11010'	1
'00011'	0	'01011'	1	'10011'	1	'11011'	0
'00100'	1	'01100'	0	'10100'	0	'11100'	1
'00101'	0	'01101'	1	'10101'	1	'11101'	0
'00110'	0	'01110'	1	'10110'	1	'11110'	0
'00111'	1	'01111'	0	'10111'	0	'11111'	1

Hausaufgabe 4 (2 + 1 Punkte):

Schreiben Sie eine Funktion, welche einen String `s` erwartet und den String zurückgibt den man erhält, wenn man alle Vorkommen des Zeichens "A" aus `s` entfernt. Sie dürfen **nicht** `replace` oder `remove` verwenden und der Rumpf der Funktionen darf je nur aus einer Zeile bestehen. Lösen Sie die Aufgabe

- (a) mittels `map` (ohne `filter` zu benutzen). Nennen Sie diese Funktion `remove_A_a`.
- (b) mittels `filter` (ohne `map` zu benutzen). Nennen Sie diese Funktion `remove_A_b`.

Hausaufgabe 5 (5 Punkte):

In dieser Aufgabe sollen Sie eine Funktion `solve_labyrinth` schreiben, welche den Weg aus einem Labyrinth findet. In diesem Labyrinth kann man an jeder Verzweigung wählen entweder nach rechts ("R") oder nach links ("L") zu gehen. Außerdem kann man davon ausgehen, dass man nach spätestens 23 Abzweigungen den Ausgang erreicht, wenn man den richtigen Weg gewählt hat. Die Lösung soll als Liste von "R" und "L" dargestellt und zurückgegeben werden.

Das Labyrinth selbst ist unbekannt, der korrekte Weg wird zufällig bestimmt und im Hauptprogramm unter der Variablen `loesung` gespeichert (ebenfalls als Liste von "R" und "L"). Dieser Teil des Programms steht Ihnen in der Datei `b10a5.py` zur Verfügung. Auf die Variable `loesung` dürfen Sie **nur** in Anweisungen der Form `L==loesung` zugreifen, wobei L irgendeine Liste ist.

Nutzen Sie **Backtracking** um die Aufgabe zu lösen. Sie dürfen der Funktion `solve_labyrinth` Hilfsparameter übergeben, um das Backtracking zu ermöglichen. Sonst erhält die Funktion `solve_labyrinth` keine Eingaben. Geben Sie auch den Funktionsaufruf von `solve_labyrinth` an, der bei Ausführung `loesung` findet und zurückgibt.

Präsenzaufgaben

Präsenzaufgabe 1:

Schreiben Sie eine Funktion `join_lists`, welche zwei Listen von Ganzzahlen erwartet. Zurückgegeben werden soll eine neue Liste, welche alle Elemente beider Listen, in aufsteigender Reihenfolge und ohne Wiederholungen, enthält. Die dürfen davon ausgehen, dass die Eingabelisten selbst keine Duplikate enthalten. Lösen Sie die Aufgabe

1. mittels List-Comprehensions (ohne Mengen zu verwenden).
2. mittels Mengen.

In beiden Fällen soll der Rumpf der Funktion nur aus einer Zeile bestehen.

Präsenzaufgabe 2:

Definieren Sie die folgenden Funktionen. Der Rumpf jeder der Funktionen darf dabei nur aus einer Zeile bestehen. Nutzen Sie je einen möglichst lesbaren List-Comprehension Ausdruck.

- (a) Die Funktion `count_a` erwartet einen String, bestehend aus Wörtern, die jeweils durch genau ein Leereichen getrennt sind. Zurückgegeben wird eine Liste mit der Anzahl der Vorkommen des Buchstaben A pro Wort (Klein- und Großschreibung).

Beispiel: `count_a("ein paar Ameisen")` liefert die Rückgabe `[0, 2, 1]`

Hinweis: Für einen String `s` und ein Zeichen `z`, ergibt die Anweisung `s.count(z)` die Anzahl der Vorkommen von `z` in `s`. Zum Beispiel wertet sich `"abaabA".count("a")` zu `3` aus.

- (b) Die Funktion `div_3_5_list` erwartet einen positiven Integer `i` und gibt die Liste aller Zahlen zwischen 1 und `i`, die durch 3 oder 5 teilbar sind, zurück.
- (c) Die Funktion `replace_minus` erwartet eine Liste von Integer-Werten und gibt die Liste zurück die resultiert, wenn man alle negativen Zahlen durch 0 ersetzt.

Präsenzaufgabe 3:

Benutzen Sie für diese Aufgabe unter anderem die Funktionen `filter` und `map`.

Hinweis: Für ein Wörterbuch `wb` liefert `wb.items()` ein iterierbares Objekt mit allen Schlüssel-Wert Paaren in `wb`, welche als Tupel dargestellt werden. Umgekehrt liefert der Befehl `dict` für einen Iterator mit Schlüssel-Wert Tupeln das entsprechende Wörterbuch.

- (a) Schreiben Sie eine Funktion `remove_palindrome`, die ein Wörterbuch `wb` mit String-Werten erwartet. Die Funktion soll aus `wb` alle Schlüssel-Wert Paare, deren Wert ein Palindrom ist, herausfiltern und das Restwörterbuch zurückgeben.

Der Rumpf der Funktion darf dabei nur aus einer Zeile bestehen.

Versuchen Sie auch die Aufgabe zu lösen, indem Sie eine rekursive Funktion an den Filter übergeben.

Ein Palindrom ist eine Zeichenkette die vorwärts und rückwärts gelesen gleich ist. Zum Beispiel "abba".

- (b) Schreiben Sie eine Funktion `dictionary_smaller`, welche zwei Wörterbücher mit Integer-Werten und genau der selben Menge an Schlüsseln, erwartet. Zurückgegeben werden soll ein neues Wörterbuch, das für jeden der Schlüssel die kleinere Zahl der beiden Wörterbücher enthält.

Beispiel:

```
dictionary_smaller({"abba":5, 12:7, -3.14:-3, -36:-1},
                  {12:10, "abba":2, -3.14:-3, -36:5})
```

liefert als Rückgabe das Wörterbuch `{"abba":2, 12:7, -3.14:-3, -36:-1}`.

- (c) Schreiben Sie eine Funktion `custom_op`, welche als Eingabe ein Wörterbuch `wb` erwartet.

Um zu beschreiben, was die Funktion leisten soll, schauen wir uns zunächst deren Einsatzbereich an. Die Funktion soll innerhalb der `map`-Operation wie folgt benutzt werden:

```
dict(map(custom_op(wb), wb.items()))
```

und folgendes leisten: Es wird überprüft, ob die Werte des Wörterbuchs

- (1) nur aus Integern bestehen oder
- (2) nur aus Strings bestehen oder
- (3) aus mehreren oder anderen Typen bestehen.

und zu

- (1) ermöglicht, dass durch die `map`-Operation alle Werte im Wörterbuch `wb` verdoppelt werden;
- (2) ermöglicht, dass durch die `map`-Operation alle Werte vom Wörterbuch `wb` gespiegelt werden;
- (3) das Wörterbuch `wb` unverändert lässt.

Hinweis: Ihre Funktion sollte einen lambda-Ausdruck, der von `wb` abhängt, zurückgeben.

Präsenzaufgabe 4:

- (a) Schreiben Sie eine Funktion `summands`, welche eine nicht-leere Liste positiver Ganzzahlen `candidates` und eine Ganzzahl `total` erwartet. Die Funktion soll eine Teilliste von `candidates` zurückgeben, deren Elemente sich zu `total` aufsummieren.

Falls solch eine Teilliste nicht existiert, soll `None` zurückgegeben werden. Falls mehrere solche Teillisten existieren, darf eine beliebig zurückgegeben werden.

Lösen Sie die Aufgabe mittels **Backtracking**.

Beispiele:

<code>summands([1, 1, 1, 1], 5)</code>	→ <code>None</code>
<code>summands([2, 3, 3], 6)</code>	→ <code>[3, 3]</code>
<code>summands([2, 3], 6)</code>	→ <code>None</code>
<code>summands([10, 45, 2, 63, 7, 2, 4], 62)</code>	→ <code>[10, 45, 7]</code>
<code>summands([2, 2, 4], 4)</code>	→ <code>[2, 2]</code> oder <code>[4]</code>
<code>summands([10, 12, 20, 22], 32)</code>	→ <code>[10, 22]</code> oder <code>[12, 20]</code>

- (b) Passen Sie Ihre Funktion an, sodass in dem Fall, dass mehrere Teillisten existieren, deren Elemente sich zu `total` aufsummieren, statt einer beliebigen, die kürzeste dieser Teillisten zurückgegeben wird. Falls mehrere Teillisten gleich lang sind, darf eine beliebige ausgewählt werden.